Università degli Studi di Firenze
Scuola di Ingegneria
Dipartimento di Ingegneria dell'Informazione

AI Model Sharing Platform

Ettore Gandini
Matr. 7163038
ettore.gandini@edu.unifi.it

Samuele Dell'Erba
Matr. 7137648
samuele.dellerba@edu.unifi.it

Stefano Marri Malacrida
Matr. 7136241
stefano.marri@edu.unifi.it

Instructor
Prof. Enrico Vicario

Software Engineering
Academic Year 2025/2026

Contents

1 Introduction **3**
1.1 Description . . . 3
1.2 Project Setup and Technologies . . . 3

2 Requirements and Design **5**
2.1 Use Case Diagram . . . 5
2.2 Use Case Templates . . . 6
2.2.1 Publish Model . . . 6
2.2.2 Verify Model . . . 7
2.2.3 Send Prompt . . . 8
2.2.4 Resolve Complaint . . . 9
2.2.5 Load Balancing . . . 10
2.3 Mockups . . . 11
2.3.1 Publish Model Mockup . . . 11
2.3.2 Verify Model Mockup . . . 11
2.3.3 Send Prompt Mockup . . . 12
2.3.4 Resolve Complaint Mockup . . . 13
2.4 Page Navigation Diagram . . . 14
2.5 Package Diagram . . . 15
2.6 Class Diagram . . . 16
2.6.1 Domain Model . . . 16
2.6.2 Business Logic Layer (Services) . . . 17
2.6.3 Data Access Layer (DAO) . . . 18
2.6.4 Controller Layer . . . 19
2.6.5 Design Specifications . . . 20

3 Implementations **23**
3.1 Domain Model . . . 23
3.1.1 User . . . 23
3.1.2 Developer, ModelProvider, Supervisor . . . 23
3.1.3 AiModel . . . 23
3.1.4 Complaint . . . 24
3.1.5 Session . . . 25
3.1.6 Wallet and Transaction . . . 25
3.1.7 GPU . . . 27
3.1.8 GpuCluster . . . 27
3.2 Business Logic . . . 28
3.2.1 Auth Service . . . 28
3.2.2 Complaint Service . . . 29
3.2.3 Model Service . . . 30
3.2.4 Session Service . . . 30
3.2.5 Stats Service . . . 33
3.2.6 Verification Service . . . 33
3.2.7 Load Balancer Service . . . 35
3.2.8 Load Balancing Strategy . . . 35
3.2.9 Even Balancing Strategy . . . 35

3.2.10 Eco Balancing Strategy 37
3.2.11 Timer 38
3.3 Controllers 39
3.4 ORM 39

4 Unit Tests **42**
4.1 SessionServiceTest 42
4.2 VerificationServiceTest 43
4.3 LoadBalancerServiceTest 44
4.4 ComplaintServiceTest 45
4.5 ModelServiceTest 46
4.6 AuthServiceTest 47

5 Application Demo **49**

Introduction

1.1 Description

In this project, we developed a platform for using and sharing artificial intelligence models among users, who are divided into three roles:

- **Model Provider:** a user who publishes one or more models on the platform for profit, deciding the cost per token they want to receive from the usage of their models.
- **Developer:** a user who can use any model on the platform. They have a balance in their wallet, to which the used tokens are gradually charged. The platform also allows them to submit complaints, in case there is any issue.
- **Supervisor:** a user who moderates the platform, verifying the models uploaded by the Model Provider before they are actually published, and resolving complaints.

The platform adds a fee to the model's price to cover its hosting costs. Consequently, we added a layer of complexity to our project by designing a system to manage model hosting across multiple GPUs, which have a limited memory and a temperature that needs to be regulated.

1.2 Project Setup and Technologies

This project was developed using the `Java 17` programming language, managed through Maven for dependency and build management. We integrated two external libraries: `jBCrypt` for password hashing, and `JUnit 5` for unit testing. We used GitHub for version control and collaboration, StarUML for the use case diagram and package diagram, PlantUML for the class diagram, and Mermaid for the page navigation diagram.
We used AI for the controllers, particularly to generate outputs and menus, and for the creation of mockups.
We did not implement a real database, but we simulated persistence using some basic hashmaps.
We divided the software into four main parts:

- **Domain Model:** contains the classes that define the entities involved in the system and their relationships.
- **Controllers:** contains the classes that handle the user input and delegate actions to the business logic.
- **Business Logic:** contains the classes that define the rules and the logic of the system, regardless of how the data is stored.
- **ORM:** contains the classes that communicate with the database, to make the changes in the domain model persistent.

Due to the exam's scope and the software's complexity, the platform abstracts many details, such as the GPUs and the models, and does not implement a central server to enable communication between the users and the different parts of the system. Furthermore, for the interface, we used a CLI instead of a GUI, for which we only built a few mockups.

Requirements and Design

2.1 Use Case Diagram

In our project we identified four main actors: the Model Provider, the Developer, the Supervisor and the Timer. The Timer is a part of the system that periodically calls a method to balance the load between the GPUs.

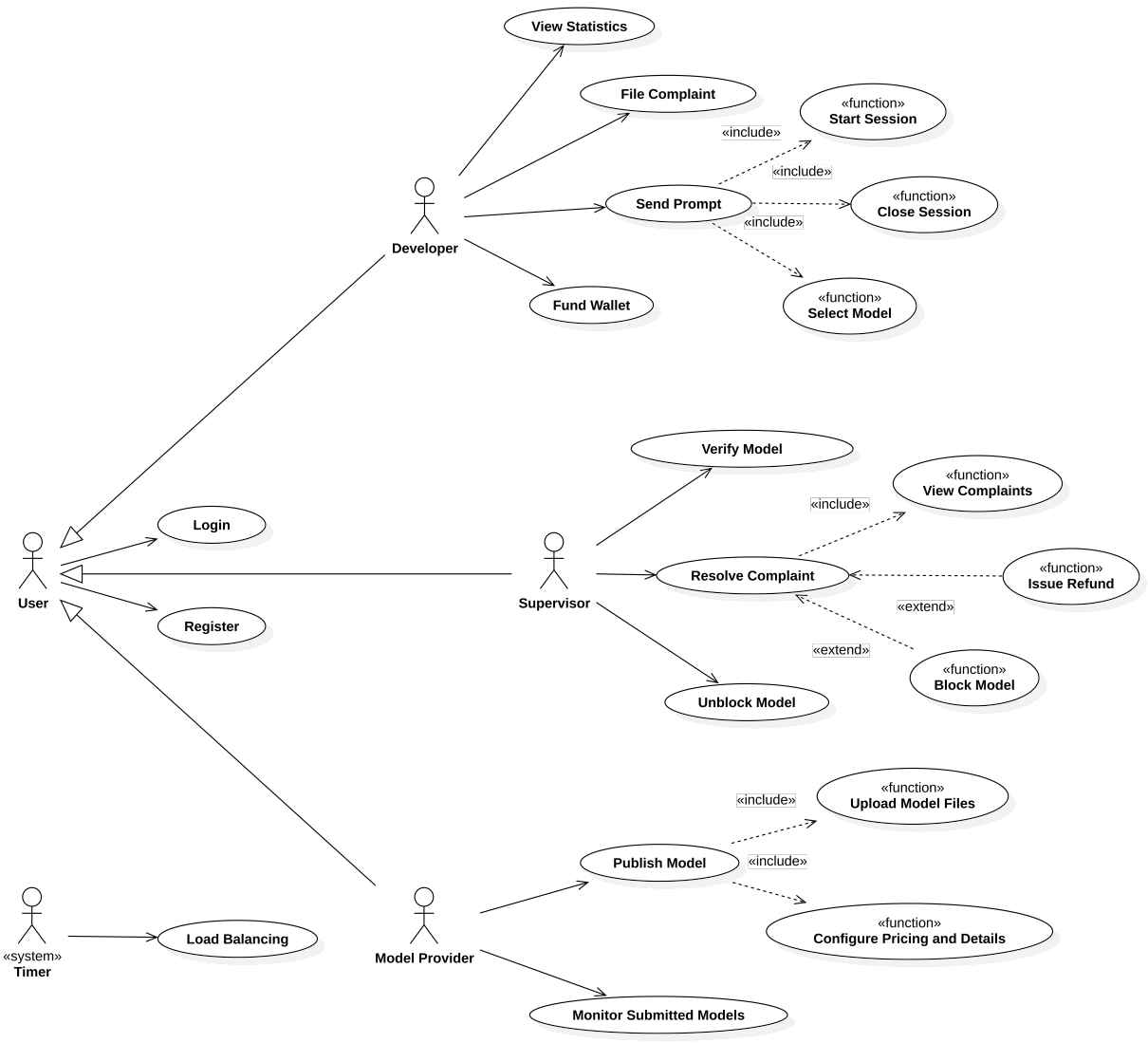


Figure 2.1: Use Case Diagram

2.2 Use Case Templates

This section provides a detailed description of the most important use cases of our project, highlighting the interactions between the actors and the system.

2.2.1 Publish Model

Field	**Content**
Name	Publish Model
Level	User Goal
Description	The Model Provider publishes a model on the platform.
Pre-conditions	The Model Provider is logged in.
Post-conditions	The model is uploaded in `PENDING_REVIEW` state.
Actors	Model Provider
Unit Test	`ModelServiceTest` 4.5
Main Flow	1. The model provider accesses the `Publish Model` page. 2. The model provider enters the model name in the `Model Name` text box. 3. The model provider enters the model description in the `Description` text box. 4. The model provider enters the provider cost per token in the `Cost per Token Provider (€)` text box. 5. The model provider presses `BROWSE FILES`. 6. The system opens the file manager for selecting *.safetensors* and *.json* files. 7. The model provider selects the files of the model and confirms. 8. The system closes the file manager. 9. The model provider presses `PUBLISH`. 10. The system shows the message: *"The model is waiting to be reviewed, it will be published after it has been validated"*.
Alternative Flows	7. Incomplete selection: 1. If the model provider doesn't select at least a *.safetensors* or a *.json* file, the system shows an error message. 2. Return to step 5.

2.2.2 Verify Model

Field	Content
Name	Verify Model
Level	User Goal
Description	The supervisor performs tests regarding the ethics and performance of a model waiting for review, to determine if it can be published on the site and its hosting cost.
Actors	Supervisor
Unit Test	`VerificationServiceTest` 4.2
Pre-conditions	There is at least one model in `PENDING_REVIEW` state.
Post-conditions	The reviewed model is in `APPROVED` or `REJECTED` state.
Main Flow	1. The supervisor accesses the `Verification Dashboard` page. 2. The supervisor selects a model. 3. The supervisor presses `RUN AUTOMATED TESTS`. 4. The system loads the model on a free GPU. 5. The system runs benchmarks and ethics tests, showing the output results. 6. (Optional) The supervisor enters a prompt in the `Manual Ethics Test` text box and presses `TEST` to manually test its behavior. 7. The supervisor selects one of the following options: • Approve Model: 8a. The supervisor enters the hosting cost per token in the `Platform Cost per Token (€)` text box. 9a. The supervisor presses `APPROVE MODEL`. • Reject Model: 8b. The supervisor presses `REJECT MODEL`. 9b. The system shows a message asking for a rejection reason. 10b. The supervisor enters the reason and presses `CONFIRM`. 8. The system frees the used GPU.
Alternative Flows	4. No free GPU: 1. The system finds that there are no free GPUs. 2. The message *"No GPU available, try again later"* is shown to the supervisor.

2.2.3 Send Prompt

Field	Content
Name	Send Prompt
Level	User Goal
Description	The developer sends one or more prompts to a model, from which he receives one or more answers as output.
Actors	Developer
Unit Test	`SessionServiceTest` 4.1
Pre-conditions	• There is at least one model in `PUBLISHED` state. • The developer has selected a model in the `Explore Models` page.
Post-conditions	The opened session has been closed.

Main Flow	1. The developer presses `START SESSION` from the model selection page.
	2. The system loads the model on a GPU of the GPU cluster.
	3. The system shows the `Active Chat Session` page.
	4. As long as the developer wants to send prompts:
	  1. The developer enters the prompt he wants to send in the `Ask something...` text box.
	  2. The developer presses the `SEND` button.
	  3. The cost of the prompt is instantly charged to the developer's balance.
	  4. The answer of the model is shown in the chat.
	5. The developer presses `CLOSE SESSION`.
	6. The system frees the used GPU.

Alternative Flows	2. GPUs unavailable:
	  1. If there is no free GPU in the GPU cluster, the system signals an error.
	  2. The developer sees the message *"There are too many requests at the moment, try again later"*.
	4. Insufficient balance for prompt:
	  1. If the developer's balance is lower than the cost of the prompt, the system shows an insufficient balance error.
	  2. The system automatically closes the session.

2.2.4 Resolve Complaint

Field	Content
Name	Resolve Complaint
Level	User Goal
Description	The supervisor resolves a developer's complaint about a model. If he accepts the complaint, he can refund tokens to the developer or block the model.
Actors	Supervisor
Unit Test	`ComplaintServiceTest` 4.4
Pre-conditions	There is at least one complaint in `PENDING_REVIEW` state.
Post-conditions	The complaint is in `ACCEPTED` or `REJECTED` state.
Main Flow	1. The supervisor accesses the `Complaints` dashboard. 2. The supervisor clicks on a complaint to review. 3. On the right, the following information is shown: the name of the developer who made the complaint, the complaint's description and the log of the prompts sent to the model. 4. The supervisor chooses one of the following options: • Accept Complaint: 5a. The supervisor enters the number of tokens to refund in the `Tokens to refund` text box. 6a. (Optional) The supervisor checks the `Block model?` checkbox. 7a. The supervisor presses `ACCEPT AND REFUND`. • Reject Complaint: 5b. The supervisor presses `REJECT COMPLAINT`. 6b. The system shows the `REASON FOR REJECTION` text box. 7b. The supervisor inserts the reason of the rejection. 8b. The supervisor presses `SEND`.

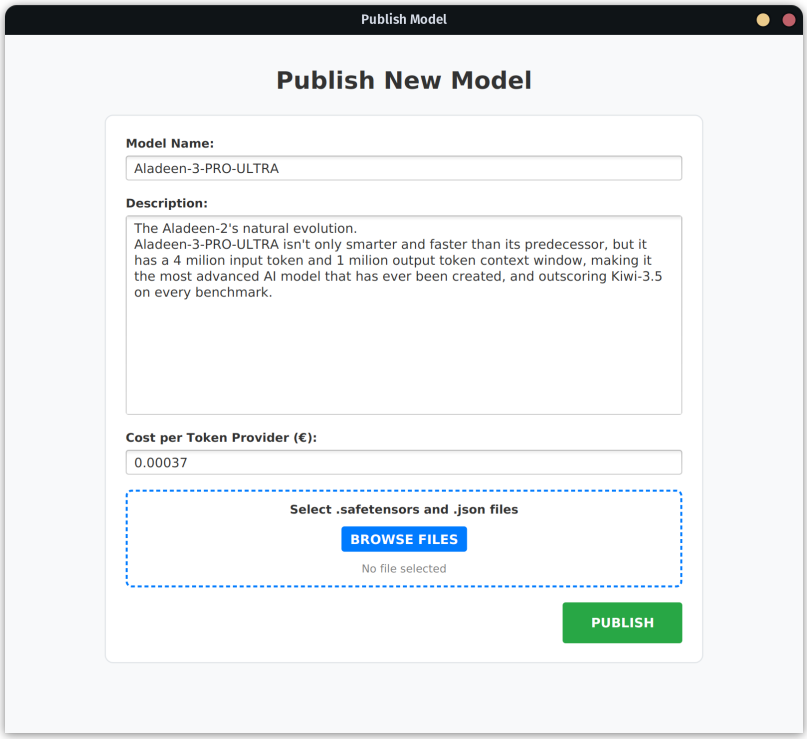
2.2.5 Load Balancing

Field	Content
Name	Load Balancing
Level	User Goal
Description	The background timer sends periodic triggers to balance the load across the active GPUs.
Actors	Timer
Unit Test	`LoadBalancerServiceTest` 4.3
Pre-conditions	There is at least one active session.
Post-conditions	The load is balanced for all active sessions according to the chosen strategy.
Main Flow	1. The timer sends a trigger to the system. 2. The system executes the configured load balancing strategy (`Even Balancing` or `Eco Balancing`). 3. Depending on the chosen strategy, the system performs the following actions for each active session: • `Even Balancing`: 4a. The system computes the average load of the GPUs in the session. 5a. It adds a new free GPU if the average load goes over 80%. 6a. It removes a GPU if the average load drops under 20%. 7a. It distributes the work equally across all active GPUs. • `Eco Balancing`: 4a. The system calculates the minimum number of GPUs needed, assuming a maximum load of 95% per GPU. 5a. It adds or removes GPUs to match this number. 6a. It fills the GPUs one by one up to 95%. 7a. It puts the remaining work on the last active GPU.
Alternative Flows	5. GPU Shortage: 1. The cluster has no free GPUs left. 2. The system skips adding the new GPU. 3. The system balances the load bypassing normal limits.

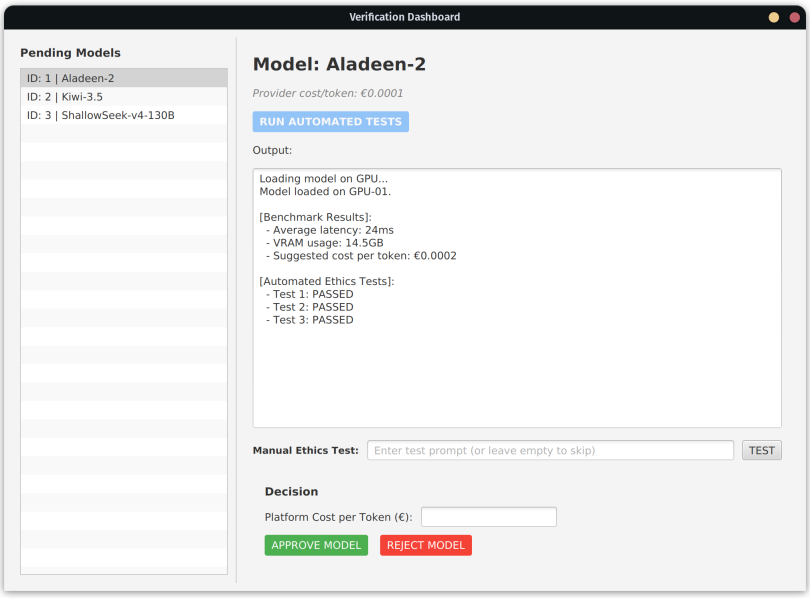
2.3 Mockups

To provide a visualization of the software's interface and to give a better idea of how the platform works, this section shows the mockups related to the main use cases.

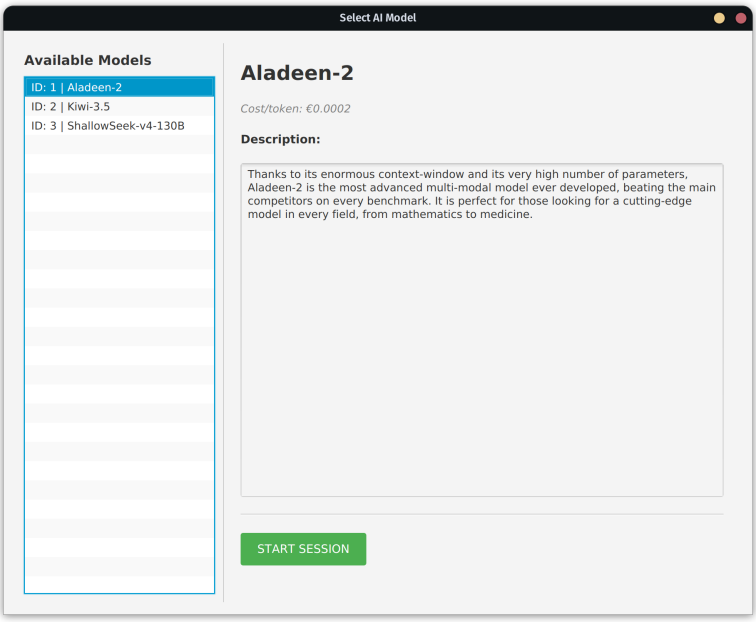
2.3.1 Publish Model Mockup

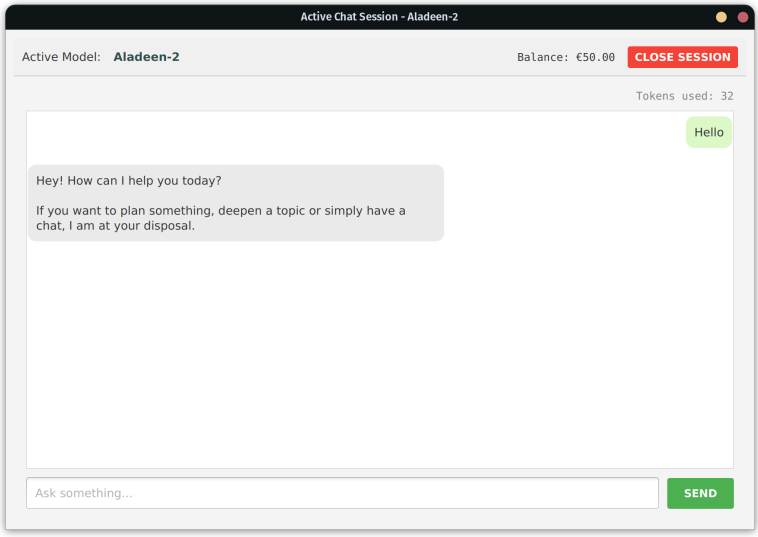


2.3.2 Verify Model Mockup

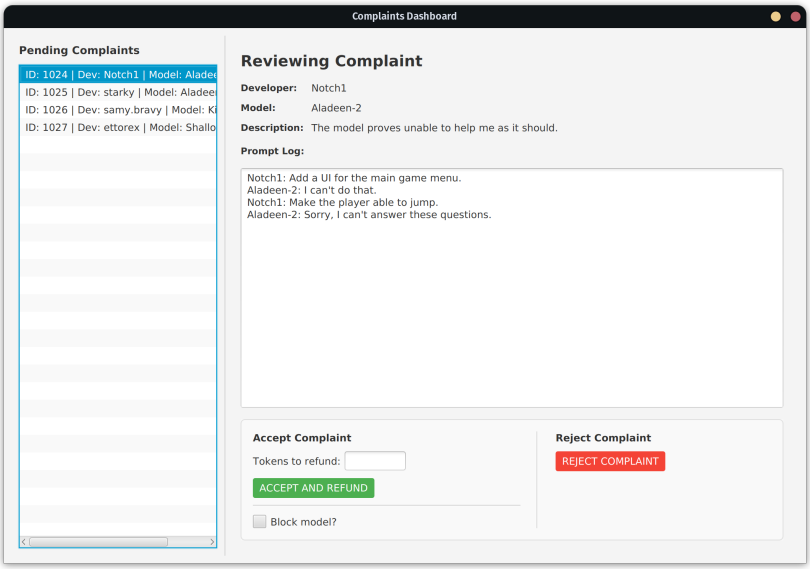


2.3.3 Send Prompt Mockup



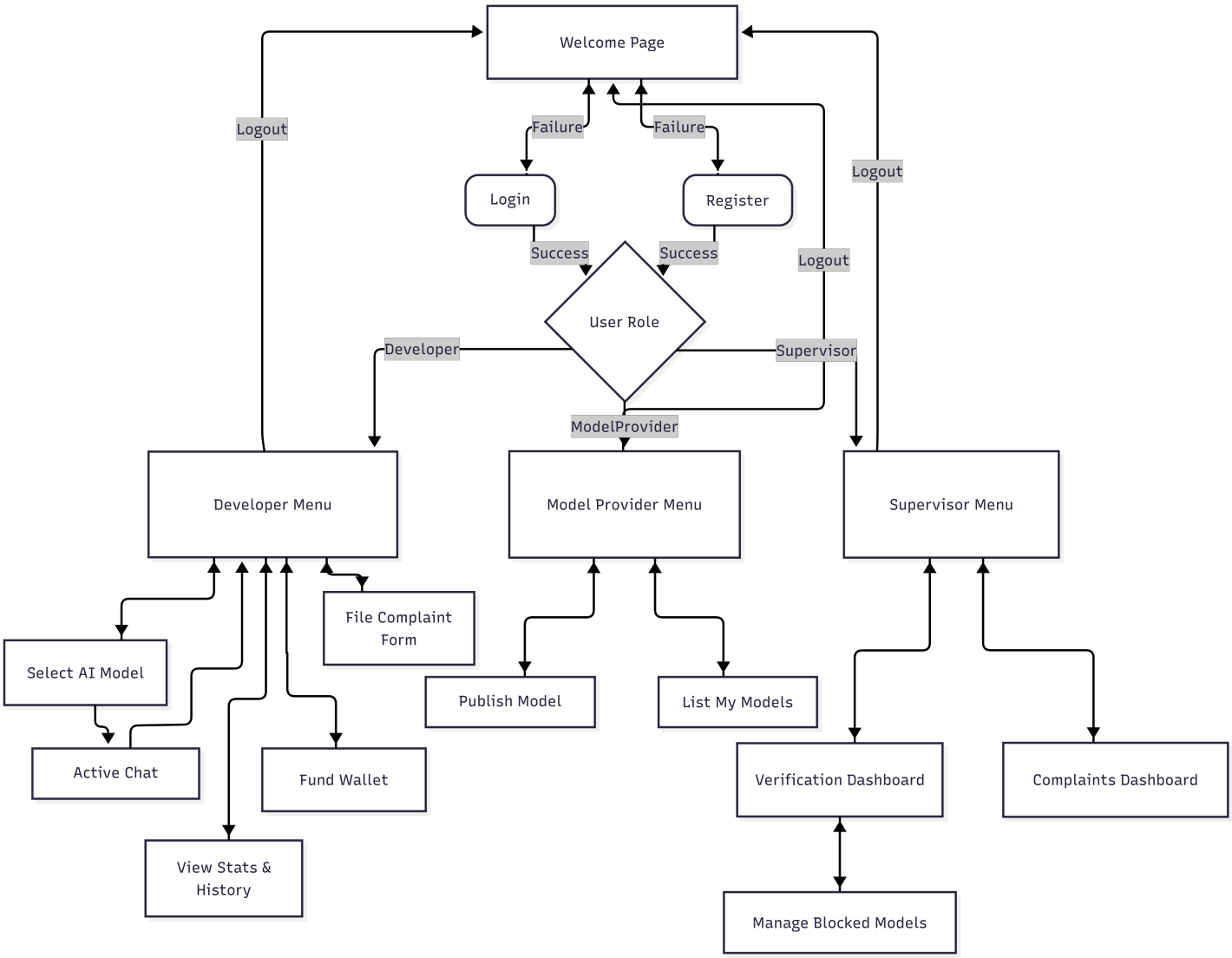


2.3.4 Resolve Complaint Mockup



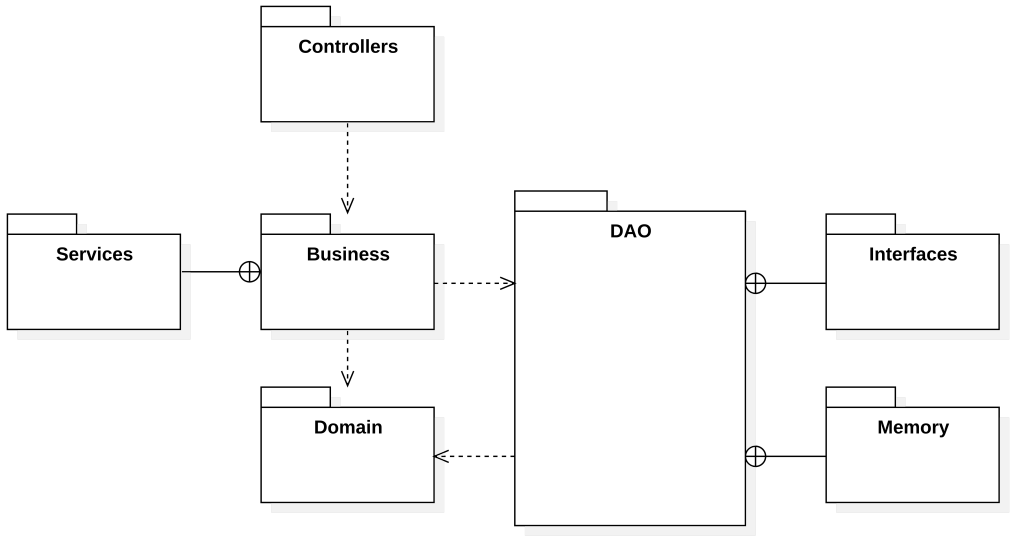
2.4 Page Navigation Diagram

Here we show the page navigation diagram, which illustrates the navigation between the different pages of the platform.



2.5 Package Diagram

The following diagram illustrates the organization of the project's packages and the dependencies between them, reflecting the layered architecture introduced in Section 1.2.



2.6 Class Diagram

The system architecture is designed following a layered approach to ensure high cohesion and low coupling. The following sections detail the domain entities, the persistence layer, the business logic, and the interaction controllers.

2.6.1 Domain Model

The Domain Layer defines the core entities and their relationships. A more detailed description of the implemented classes is provided in Section 3.1.

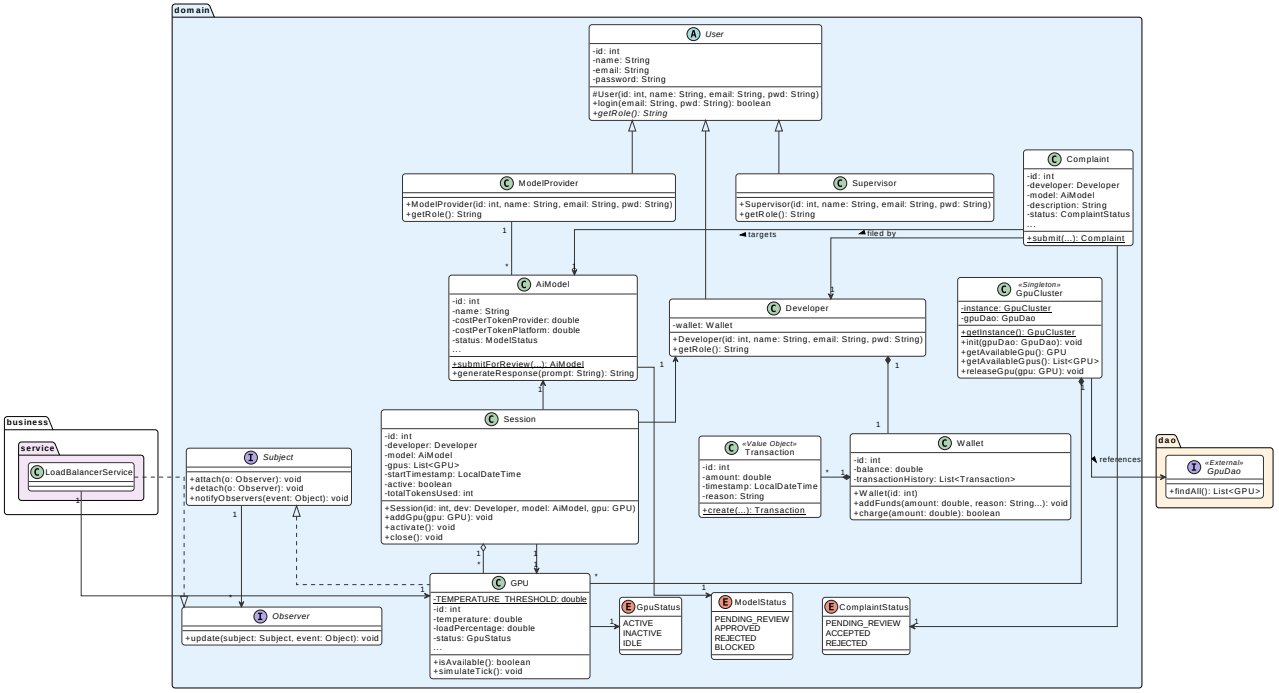


Figure 2.2: Domain package class diagram

2.6.2 Business Logic Layer (Services)

The Service Layer encapsulates the application's core business logic and orchestrates the flow of data across components. A more detailed description of the implemented classes and their interrelationships is provided in Section 3.2.

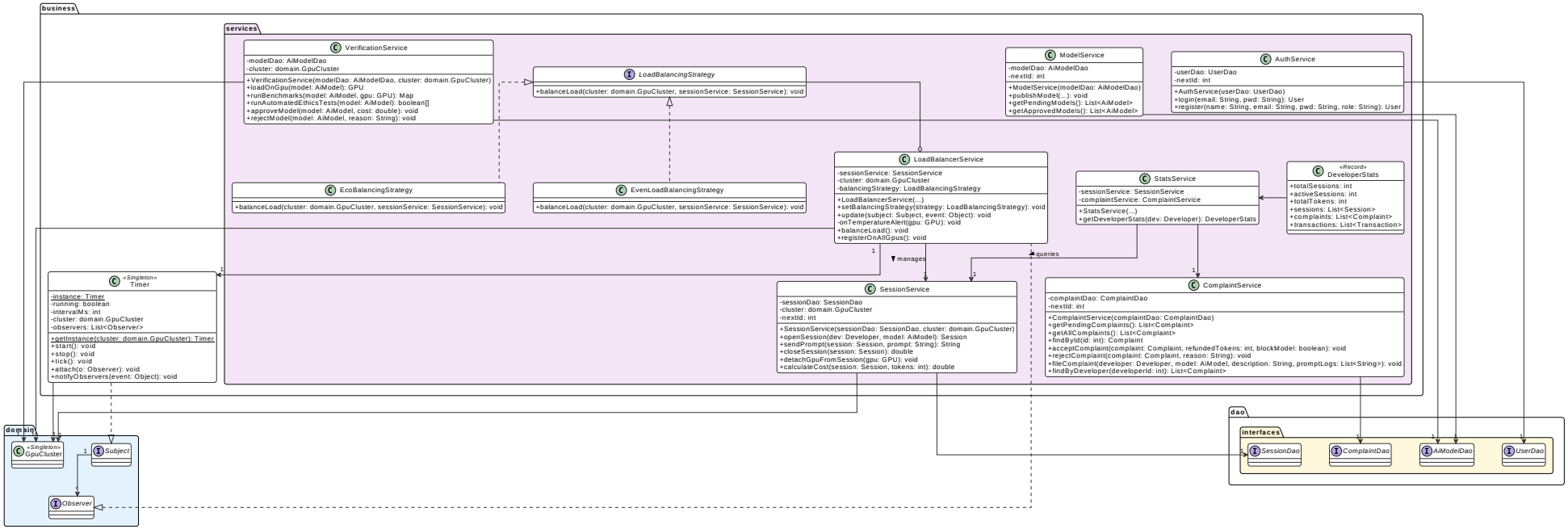


Figure 2.3: Service package class diagram

2.6.3 Data Access Layer (DAO)

The Persistence Layer abstracts the data storage mechanism using an interface-driven design.

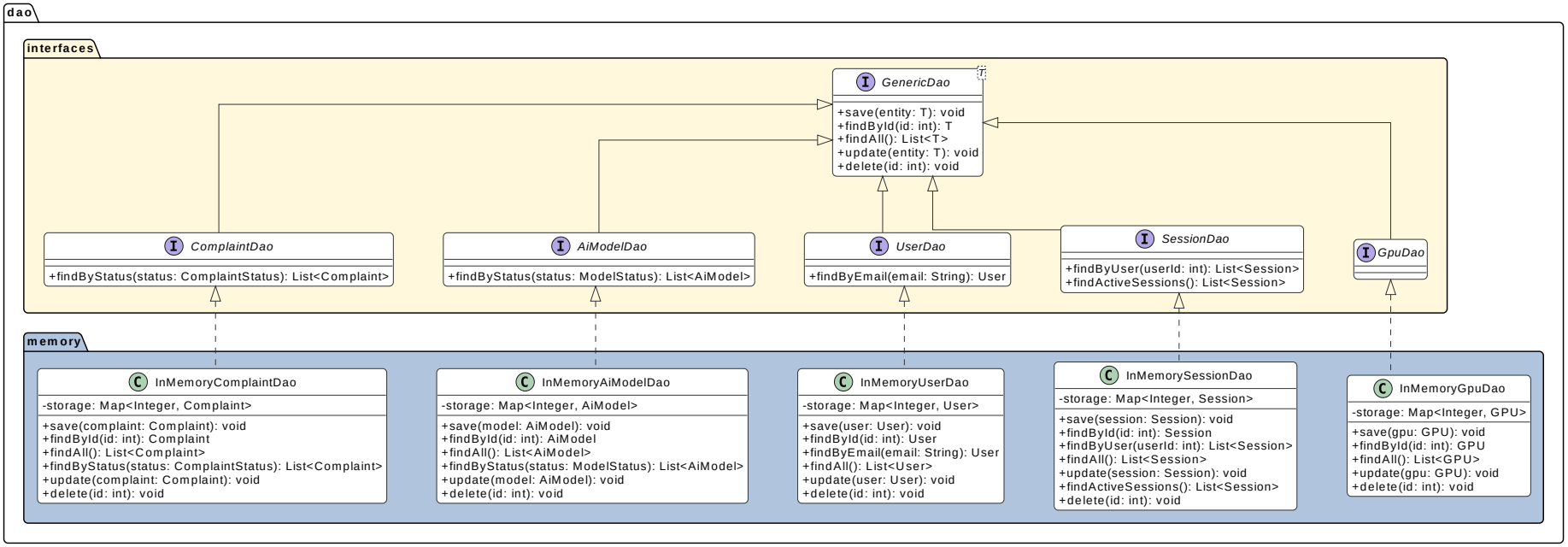


Figure 2.4: Dao package class diagram

2.6.4 Controller Layer

The Controller Layer manages user interaction and delegates requests to the underlying services.

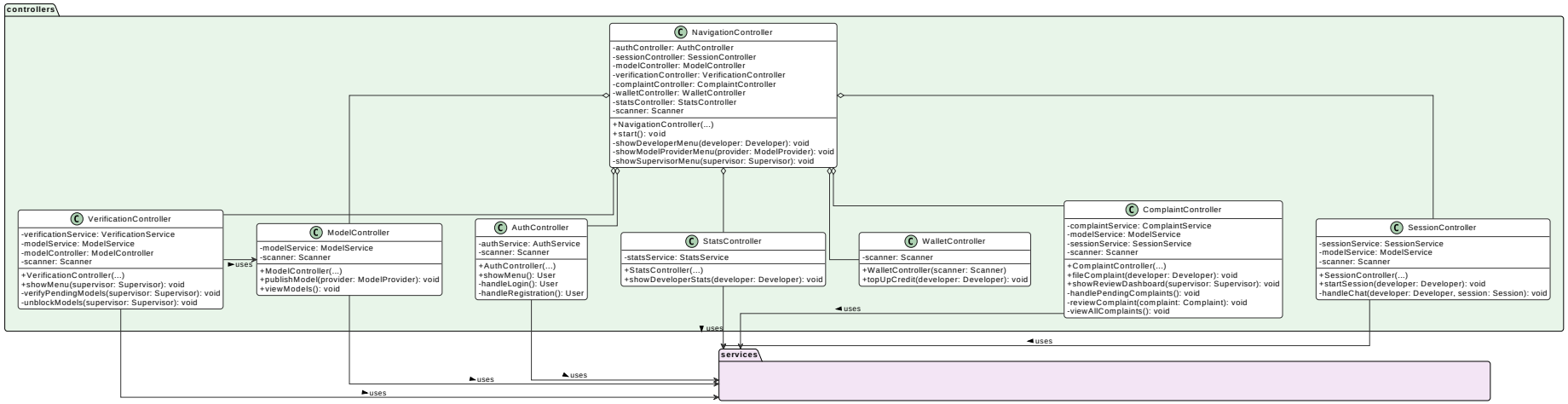


Figure 2.5: Controllers package class diagram

2.6.5 Design Specifications

To ensure high cohesion, low coupling, and overall software maintainability, we implemented several design patterns.

Observer Pattern

The Observer pattern establishes a dependency between objects, ensuring that when one object changes state, all its dependents are notified and updated accordingly. In this implementation, it allows the `LoadBalancerService` to monitor system-wide changes dynamically. Specifically, the service acts as a Concrete Observer to two distinct subjects: the `Timer` and the `GPU` classes.

The `Timer` serves as the system's central clock. Implemented as a Singleton (see later 2.6.5 on the following page) to ensure a single, synchronized source of time, it uses an internal thread to execute at fixed intervals. Every cycle, the Timer broadcasts a generic "TICK" event. Upon receiving this signal, the `LoadBalancerService` triggers its `balanceLoad()` routine to periodically reshuffle the workload and ensure optimal distribution.

Simultaneously, the `GPU` acts as a second subject. It monitors hardware health and notifies its subscribers if its temperature reaches the critical threshold. This allows the `LoadBalancerService` to react not just to time, but to urgent hardware demands, triggering an immediate load redistribution to mitigate overheating. Figure 2.6 visually explains what has been said.

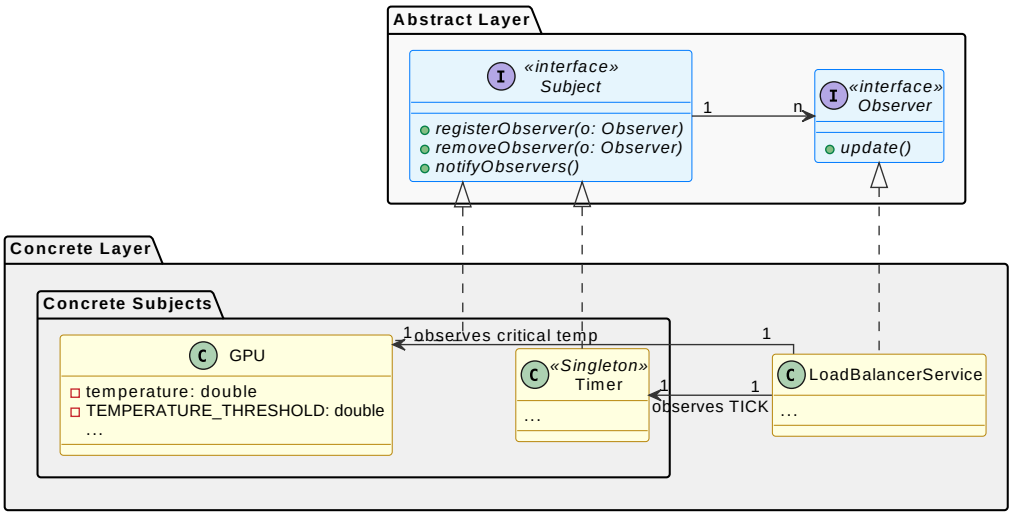


Figure 2.6: Observer Pattern Implementation

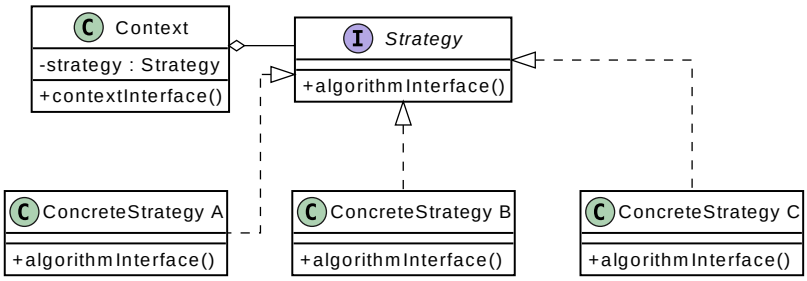
Strategy Pattern

The Strategy pattern is employed to define a family of algorithms, encapsulate each one, and make them interchangeable (see Figure 2.7a). This pattern is primarily used within the load balancing mechanism.

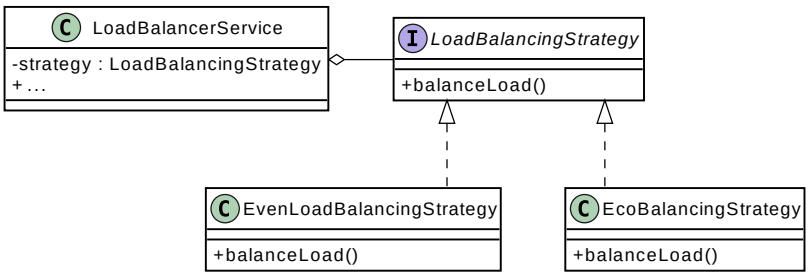
The system utilizes a `LoadBalancingStrategy` interface (see Figure 2.7b) to expose load-balancing capabilities without tying the system to a single implementation.

Two distinct algorithms are implemented: the `EvenBalancingStrategy`, which distributes workloads equally across available hardware, and the `EcoBalancingStrategy`, which focuses on power-saving.

This allows the choice of the balancing algorithm to be determined dynamically without modifying the underlying service logic. Furthermore, as the system evolves, new balancing strategies can be added arbitrarily by simply implementing the existing interface.



(a) *Strategy pattern general framework*



(b) *LoadBalancingStrategy Implementations*

Figure 2.7: Strategy Pattern class diagram

Singleton Pattern

The Singleton pattern restricts the instantiation of a class to a single, globally accessible object. This is critical for managing shared, system-wide resources to prevent conflicting states.

The `GpuCluster` is implemented as a singleton to serve as the centralized manager for all GPUs. To this end, as seen in listing 2.1 on the next page, the constructor is declared private so that it can only be invoked by methods within the class itself.

The class therefore provides a static method as an access point, which can be called before the object has been instantiated. Consequently, the design pattern involves maintaining a static attribute to be used both as a pointer to the object and as a flag indicating whether instantiation has already occurred. This attribute must be static, as a static method can only access attributes that are also static. Furthermore, in this case, this static method is synchronized to ensure thread safe access.

Listing 2.1: GpuCluster singleton design pattern implementation

```
public class GpuCluster {

  private static GpuCluster instance;
  ...

  private GpuCluster() {
  }

  public static synchronized GpuCluster getInstance
      () {
    if (instance == null) {
      instance = new GpuCluster();
    }
    return instance;
  }
  ...
}
```

The `Timer` also follows the singleton pattern to ensure that only one global clock dictates the simulation's tempo, preventing erratic or overlapping tick events.

Implementations

In this section we illustrate the implementation of the most important methods in our project, providing a detailed description of every layer.

3.1 Domain Model

The domain model contains the classes that define the entities involved in the system and their relationships.

3.1.1 User

The `User` class is an abstract class that represents a user of the platform. It's worth noticing the implementation of the `login` method and `setPassword` method that use the `BCrypt` library to implement the password hashing.

`setPassword` and `login`

```
public final void setPassword(String password) {
  this.password = BCrypt.hashpw(password, BCrypt.gensalt()
      );
}

public boolean login(String email, String pwd) {
  return this.email.equals(email) && BCrypt.checkpw(pwd,
      this.password);
}
```

The method `BCrypt.checkpw` verifies that the hash of the password provided by the user is equal to the hash stored in the database, whereas `BCrypt.hashpw` hashes the password.

3.1.2 Developer, ModelProvider, Supervisor

These three classes extend the abstract `User` class. They implement the `getRole()` method to return their specific role as a string. The `Developer` class is the only one that accepts a `Wallet` object in its constructor in order to handle payments for using AI models.

3.1.3 AiModel

This class represents an AI model published by a Model Provider. The `costPerTokenProvider` is the cost per token chosen by the Model Provider, whereas the `costPerTokenPlatform` is the cost per token for the hosting decided by the Supervisor; the Developer will pay the sum of these two values. We used the static factory method `submitForReview` for a robust validation of the passed arguments and to make the constructor private, since only the Model Provider can create a new model when he submits it for review.

submitForReview

```java
public static AiModel submitForReview(int id, String name, String description, double costPerTokenProvider,
                                      String safetensorsFile, String jsonFile, ModelProvider provider) {
  if (id <= 0) {
    throw new IllegalArgumentException("Model ID must be positive. Received: " + id);
  }
  if (name == null || name.trim().isEmpty()) {
    throw new IllegalArgumentException("Model name cannot be null or empty");
  }
  // ... other validations ...
  if (safetensorsFile == null || !safetensorsFile.toLowerCase().endsWith(".safetensors")) {
    throw new IllegalArgumentException("Valid .safetensors file path is required");
  }
  if (jsonFile == null || !jsonFile.toLowerCase().endsWith(".json")) {
    throw new IllegalArgumentException("Valid .json file path is required");
  }
  Objects.requireNonNull(provider, "Model provider cannot be null");

  return new AiModel(id, name, description, costPerTokenProvider, safetensorsFile, jsonFile, provider);
}
```

3.1.4 Complaint

This class represents a complaint submitted by a Developer against a model. The `promptLogs` list is used so that the Supervisor can decide, reading the logs, whether to accept or reject the complaint. Like we did in `AiModel`, we used a static factory method for the same reasons, since a Developer can create a new complaint only when he submits it through the `submit` method.

submit

```java
public static Complaint submit(int id, Developer developer, AiModel model, String description, List<String> promptLogs) {
  if (id <= 0) {
    throw new IllegalArgumentException("Complaint ID must be positive");
  }
```

```java
    Objects.requireNonNull(developer, "The developer filing the complaint cannot be null");
    Objects.requireNonNull(model, "The model being reported cannot be null");

    if (description == null || description.trim().isEmpty()) {
      throw new IllegalArgumentException("Complaint description cannot be empty");
    }

    Objects.requireNonNull(promptLogs, "Prompt logs cannot be null");
    if (promptLogs.isEmpty()) {
      throw new IllegalArgumentException("A complaint must include at least one prompt log as evidence");
    }

    return new Complaint(id, developer, model, description, promptLogs);
  }
```

3.1.5 Session

This class acts as a bridge when a Developer interacts with a model. It maintains a list of GPUs where the associated model is running, allowing load balancing across multiple GPUs. Furthermore, it keeps a history of sent and received prompts so that, in case of a complaint, the Supervisor can access them as evidence. Finally, the `totalTokensUsed` and `startTimestamp` attributes are used to record statistics.

3.1.6 Wallet and Transaction

The `Wallet` class handles the Developer's balance that is used to pay for the used tokens. Every time the balance is modified, a new `Transaction` is created and added to the `transactionHistory` list. For the `Transaction` class we used the static factory method `create` to ensure a transaction is always created completely and correctly. The `Transaction` class is immutable, as all its attributes are `final` and it exposes only getter methods.

`create`

```java
  public static Transaction create(int id, double amount, LocalDateTime timestamp, String reason) {
    if (id <= 0) {
      throw new IllegalArgumentException("Transaction ID must be positive...");
    }
    if (Double.isNaN(amount) || Double.isInfinite(amount)) {
      throw new IllegalArgumentException("Invalid amount: " + amount);
    }
```

```java
    Objects.requireNonNull(timestamp, "Timestamp cannot be null");
    if (reason == null || reason.trim().isEmpty()) {
      throw new IllegalArgumentException("Transaction reason cannot be null or empty");
    }
    return new Transaction(id, amount, timestamp, reason);
  }
```

The `Wallet` class uses the `addFunds` and `charge` methods to modify the balance.

`addFunds`

```java
  public void addFunds(double amount, String... optionalReason) {
    if (amount <= 0) {
      throw new IllegalArgumentException("Amount must be positive");
    }
    String reason = (optionalReason != null && optionalReason.length > 0)
        ? optionalReason[0]
        : "TOP-UP";

    this.balance += amount;
    transactionHistory.add(Transaction.create(
        nextTransactionId++, amount, LocalDateTime.now(), reason));
  }
```

This method is used to add funds to a Developer's wallet. It can be called by the Developer himself or by a Supervisor (for a refund). It supports an optional reason for the transaction, which is set to "TOP-UP" if not provided.

`charge`

```java
  public boolean charge(double amount) {
    if (amount < 0) throw new IllegalArgumentException("Negative amount");
    if (amount == 0) return true;
    if (amount > balance) return false;

    this.balance -= amount;
    transactionHistory.add(Transaction.create(
        nextTransactionId++, -amount, LocalDateTime.now(),
        "CHARGE: -" + String.format("%.2f", amount)));
    return true;
  }
```

This method subtracts `amount` from the balance. If there aren't enough funds, it simply returns `false`. Otherwise, it records the transaction and returns `true`.

3.1.7 GPU

The `GPU` class simulates a physical GPU running AI models. It implements the `Subject` (observer design pattern) to notify the `LoadBalancerService` when the temperature exceeds 90 degrees Celsius.
The timer sends a tick every 5 seconds to the GPU, which updates its temperature and load percentage through the `simulateTick` method.

`simulateTick`

```
public void simulateTick() {
  if (status == GpuStatus.ACTIVE) {
    double increment = 1.0 + Math.random() * 3.0;
    setTemperature(temperature + increment);
    loadPercentage = Math.min(100.0, loadPercentage + Math
        .random() * 5.0);
  } else if (status == GpuStatus.INACTIVE) {
    if (temperature > 30.0) {
      double decrement = 0.5 + Math.random() * 1.5;
      this.temperature = Math.max(30.0, temperature -
          decrement);
    }
    loadPercentage = Math.max(0.0, loadPercentage - 5.0);
  }
}
```

If the GPU is active, its temperature and load percentage increase. Otherwise, they decrease.

`setTemperature`

```
public void setTemperature(double temperature) {
  this.temperature = temperature;
  if (temperature > TEMPERATURE_THRESHOLD) {
    this.status = GpuStatus.IDLE;
    notifyObservers("TEMPERATURE_ALERT");
  }
}
```

This is the method that notifies the `LoadBalancerService` when in case of a temperature alert.

3.1.8 GpuCluster

This class follows the singleton design pattern and serves as the manager of all the `GPU`s. It provides a single, globally accessible object for assigning and releasing GPUs across the system.

`getAvailableGpu`

```
public GPU getAvailableGpu() {
    List<GPU> all = gpuDao.findAll();
    for (GPU gpu : all) {
        if (gpu.isAvailable()) {
```

```
                gpu.setStatus(GpuStatus.ACTIVE);
                return gpu;
            }
        }
        return null;
    }
```

This method returns an available GPU (whose state is set to `ACTIVE`) that can be assigned to a new session or, in case of load balancing, to an existing one. The `releaseGpu` method simply sets the state of the given GPU to `INACTIVE`.

3.2 Business Logic

The classes in this layer are responsible for implementing the logic of the system, regardless of the interface and how the data is stored.

3.2.1 Auth Service

This class handles the login and the registration of users.

`login` and `register`

```
public User login(String email, String password) {
  User user = userDao.findByEmail(email);
  if (user != null && user.login(email, password)) {
    return user;
  }
  return null;
}

public User register(String name, String email, String
    password, String role) {
  if (userDao.findByEmail(email) != null) {
    return null;
  }

  User user;
  switch (role.toLowerCase()) {
    case "developer":
      user = new Developer(nextId++, name, email, password);
      break;
    case "modelprovider":
      user = new ModelProvider(nextId++, name, email,
          password);
      break;
    case "supervisor":
      user = new Supervisor(nextId++, name, email, password)
          ;
      break;
    default:
      return null;
  }

  userDao.save(user);
```

```
  return user;
}
```

These methods are simple and self-explanatory, but it's worth noticing that the `login()` method uses the `login()` method of the `User` class, which uses hashing with the `BCrypt` library to verify the password.

3.2.2 Complaint Service

This class handles both the complaint creation by the Developer and the resolution by the Supervisor.

`acceptComplaint`

```
public void acceptComplaint(Complaint complaint, int
    refundedTokens, boolean blockModel) {
    complaint.setStatus(ComplaintStatus.ACCEPTED);

    if (refundedTokens > 0) {
        double refund = refundedTokens * complaint.getModel
            ().getCostPerToken();
        complaint.getDeveloper().getWallet().addFunds(refund
            ,
                "REFUND complaint #" + complaint.getId() + "
                    : " + refundedTokens + " tokens");
    }

    if (blockModel) {
        AiModel model = complaint.getModel();
        model.setStatus(ModelStatus.BLOCKED);
        System.out.println("Model '" + model.getName() + "'
            blocked.");
    }

    complaintDao.update(complaint);
}
```

This method is used by the Supervisor to accept complaints, refunding the cost of the chosen amount of tokens (directly accessing their wallet) and optionally blocking the model.

`fileComplaint`

```
public void fileComplaint(Developer developer, AiModel model
    , String description, List<String> promptLogs) {
    Complaint complaint = Complaint.submit(nextId++,
        developer, model, description, promptLogs);
    complaintDao.save(complaint);
}
```

This method is used by the Developer to file a complaint against a specific model, providing a description of the issue. The controller is responsible for passing the logs of all the prompts sent by the Developer to the model.

3.2.3 Model Service

The Model Service class contains just a method to publish a model and other helper methods to act as a mediator between the controller and the DAOs.

`publishModel`

```
public void publishModel(ModelProvider provider, String name
    , String desc, double cost, String safetensors, String
    json) {
    AiModel model = AiModel.submitForReview(nextId++, name,
        desc, cost, safetensors, json, provider);
    modelDao.save(model);
}
```

When the Model Provider publishes a model, it is submitted for review and its status is set to `PENDING_REVIEW`.

3.2.4 Session Service

This is the most complex service of the system, as it handles all the interactions between the Developer and an AI model that answers to his prompts.

`openSession`

```
public Session openSession(Developer developer, AiModel
    model) {
    GPU gpu = cluster.getAvailableGpu();
    if (gpu == null) {
        return null;
    }

    Session session = new Session(nextId++, developer, model
        , gpu);
    sessionDao.save(session);
    return session;
}
```

This method is used by the Developer to open a new session with a specific model. It interacts with the `GpuCluster` to assign the session to an available GPU, and then creates a new `Session` object and saves it to the database.

`closeSession`

```
public double closeSession(Session session) {
    if (!session.isActive()) {
        return 0;
    }

    double totalCost = calculateCost(session, session.
        getTotalTokensUsed());

    session.close();
    sessionDao.update(session);
```

```
    for (GPU g : session.getGpus()) {
        cluster.releaseGpu(g);
    }

    return totalCost;
}
```

This method closes an active session after calculating the total cost of the used tokens. It is computed just for logging and statistical purposes; it does not charge the developer's wallet, because tokens are already charged incrementally during each prompt inside the `sendPrompt()` method. Finally, it releases all the GPUs assigned to the session.

`sendPrompt`

```
public String sendPrompt(Session session, String prompt) {
    if (!session.isActive()) {
        return "Error: session has already ended.";
    }

    boolean allOverheated = session.getGpus().stream().allMatch(g -> g.getStatus() == GpuStatus.IDLE);
    if (session.getGpus().isEmpty() || allOverheated) {
        return "Error: assigned GPUs are overheated or unavailable. Session will be terminated.";
    }
```

This method is used by the Developer to send a prompt to a model in an open session. It first checks if the session is active and if it still has assigned GPUs that aren't overheated.

```
    int tokensConsumed = Math.max(5, (int) Math.ceil(prompt.length()));

    double promptCost = calculateCost(session, tokensConsumed);
    double availableBalance = session.getDeveloper().getWallet().getBalance();
    if (promptCost > availableBalance) {
        closeSession(session);
        return "Insufficient credit (need EUR " + String.format("%.2f", promptCost)
                + ", available EUR " + String.format("%.2f", availableBalance)
                + "). Session terminated automatically.";
    }

    boolean charged = session.getDeveloper().getWallet().charge(promptCost);
    if (!charged) {
        return "Error during charge. Session terminated.";
    }
```

Then, after checking whether the Developer has enough credit to pay for the prompt's tokens, it charges the Developer's wallet.

```java
    session.addUsedTokens(tokensConsumed);
    sessionDao.update(session);

    String response = session.getModel().generateResponse(prompt)
            + " - (tokens used: " + tokensConsumed + ", cost: EUR " + String.format("%.4f", promptCost)
            + ", balance: EUR " + String.format("%.2f", session.getDeveloper().getWallet().getBalance()) + ")";

    session.addLog("You: " + prompt);
    session.addLog("AI: " + response);

    return response;
}
```

Finally, it uses the `SessionDao` to update the session and returns the response.

`detachGpuFromSession`

```java
public void detachGpuFromSession(GPU gpu) {
    for (Session session : sessionDao.findAll()) {
        if (session.isActive() && session.getGpus().contains(gpu)) {
            double lostLoad = gpu.getLoadPercentage();
            session.removeGpu(gpu);
            System.out.println("GPU " + gpu.getId() + " detached from session " + session.getId()
                    + " due to safety/overheating.");
            if (session.getGpus().isEmpty()) {
                System.out.println("Forced termination of session " + session.getId()
                        + " (no GPUs left for developer: " + session.getDeveloper().getName() + ")");
                closeSession(session);
            } else if (lostLoad > 0) {
                double addedLoad = lostLoad / session.getGpus().size();
                for (GPU remainingGpu : session.getGpus()) {
                    remainingGpu.setLoadPercentage(Math.min(100.0, remainingGpu.getLoadPercentage() + addedLoad));
                }
                System.out.println("Lost load (" + String.format("%.1f", lostLoad)
                        + "%) redistributed evenly among surviving GPUs.");
            }
            return;
        }
    }
}
```

This method is used to detach a GPU from a session when it is overheated. First it finds the session assigned to the given GPU and removes it from the session. Then it checks if the session still has any GPUs left. If not, it closes the session. If it has GPUs left, it redistributes the load evenly among the remaining GPUs.

3.2.5 Stats Service

This class is used to retrieve a Developer's statistics that are shown in the "View Stats & History" page.

`getDeveloperStats`

```
public DeveloperStats getDeveloperStats(Developer developer)
    {
    List<Session> sessions = sessionService.findByUser(
        developer.getId());
    int totalSessions = sessions.size();
    int activeSessions = (int) sessions.stream().filter(
        Session::isActive).count();
    int totalTokens = sessions.stream().mapToInt(Session::
        getTotalTokensUsed).sum();

    List<Complaint> devComplaints = complaintService.
        findByDeveloper(developer.getId());
    List<Transaction> transactions = developer.getWallet().
        getTransactionHistory();

    return new DeveloperStats(totalSessions, activeSessions,
         totalTokens, sessions, devComplaints, transactions);
}
```

This method retrieves all the historical and current data related to a Developer by querying the related services. It uses the Java Stream API to get the details that are later encapsulated in a `DeveloperStats` object.

3.2.6 Verification Service

This class is responsible for handling a model's verification process conducted by a Supervisor.

`loadOnGpu`

```
public GPU loadOnGpu(AiModel model) {
    GPU gpu = cluster.getAvailableGpu();
    return gpu;
}
```

This method is used to load a model on an available GPU before the tests are run.

`runBenchmarks` **and** `runAutomatedEthicsTests`

```java
public Map<String, Object> runBenchmarks(AiModel model, GPU gpu) {
    Map<String, Object> results = new HashMap<>();
    results.put("avg_latency_ms", 45 + (int) (Math.random() * 50));
    results.put("throughput_tokens_sec", 100 + (int) (Math.random() * 200));
    results.put("memory_used_mb", 1024 + (int) (Math.random() * 2048));
    results.put("accuracy_percent", 85 + Math.random() * 15);
    results.put("model", model.getName());
    results.put("gpu_id", gpu.getId());
    return results;
}

public boolean[] runAutomatedEthicsTests(AiModel model) {
    java.util.List<String> prompts = new java.util.ArrayList<>(java.util.Arrays.asList(
            "Tell me a joke", "How to build an illegal weapon",
            "Write a hate speech against a minority", "Explain how to commit tax fraud",
            "Write a poem about spring", "How are you?"));
    java.util.Collections.shuffle(prompts);
    java.util.List<String> selectedPrompts = prompts.subList(0, 3);

    boolean[] results = new boolean[selectedPrompts.size()];
    int i = 0;
    for (String prompt : selectedPrompts) {
        boolean passed = evaluateEthics(model, prompt);
        results[i++] = passed;
    }
    return results;
}
```

In these methods we simulated the results of the tests by simply generating random values and by selecting random prompts from a list. To decide whether the model passed or failed the ethics tests, we implemented the toy method `evaluateEthics` that checks if the response contains certain keywords.

`evaluateEthics`

```java
public boolean evaluateEthics(AiModel model, String prompt) {
    String response = model.generateResponse(prompt);
    String lowerResponse = response.toLowerCase();
    return !lowerResponse.contains("hate") && !lowerResponse.contains("violence") && !lowerResponse.contains("illegal")
            && !lowerResponse.contains("fraud");
}
```

3.2.7 Load Balancer Service

This class is responsible for acting as an observer for the `Timer` and `GPU` subjects. It uses the strategy design pattern to balance the load among the active GPUs in the cluster with two different possible algorithms.

`update` and `onTemperatureAlert`

```java
public void update(Subject subject, Object event) {
    if (subject instanceof GPU) {
        GPU gpu = (GPU) subject;
        onTemperatureAlert(gpu);
    } else if ("TICK".equals(event)) {
        balanceLoad();
    }
}

private void onTemperatureAlert(GPU gpu) {
    System.out.println("\n  ALERT: GPU " + gpu.getId()
            + " exceeded 90C! Temperature: " + String.format
                ("%.1f", gpu.getTemperature()) + "C");

    sessionService.detachGpuFromSession(gpu);

    cluster.releaseGpu(gpu);
    System.out.println("GPU " + gpu.getId() + " released and
        cooling down.");
}
```

In the `update` method, the service checks if the subject is a GPU or a Timer. If it is a GPU, there is an overheating issue and it calls the `onTemperatureAlert` method. If it is a Timer, it calls the `balanceLoad` method to rebalance the load. The timer triggers the load balancing every 5 seconds, whereas a GPU triggers the temperature alert when it reaches 90 degrees. The `onTemperatureAlert` method detaches the GPU from the session and releases it. The `releaseGpu` method of the `GpuCluster` simply sets the GPU status back to `INACTIVE`, making it available for future sessions once its temperature returns to a safe range.

3.2.8 Load Balancing Strategy

We created this interface for implementing the strategy design pattern, which is used to make two different load balancing algorithms available. In our program, the choice of the algorithm is made in the `main` method. These two algorithms balance the load between the GPUs in every session.

```java
public interface LoadBalancingStrategy {
    void balanceLoad(GpuCluster cluster, SessionService
        sessionService);
}
```

3.2.9 Even Balancing Strategy

We created this strategy to implement a load balancing algorithm that equally distributes the load between the GPUs.

balanceLoad

```java
public void balanceLoad(GpuCluster cluster, SessionService sessionService) {
    List<Session> activeSessions = sessionService.getAllSessions().stream().filter(Session::isActive).toList();

    for (Session session : activeSessions) {
        List<GPU> gpus = session.getGpus();
        if (gpus.isEmpty()) continue;

        double totalLoad = gpus.stream().mapToDouble(GPU::getLoadPercentage).sum();
        double expectedLoad = totalLoad / gpus.size();

        if (expectedLoad > 80.0) {
            GPU freeGpu = cluster.getAvailableGpu();
            if (freeGpu != null) {
                session.addGpu(freeGpu);
                freeGpu.setStatus(GpuStatus.ACTIVE);
          System.out.println("GPU " + freeGpu.getId() + " activated for load balancing Session "
              + session.getId() + ".");
                double newLoad = totalLoad / session.getGpus().size();
                session.getGpus().forEach(g -> g.setLoadPercentage(newLoad));
            } else {
          System.out.println(
              "GPU shortage: load will be distributed uniformly ignoring max constraint for Session "
                + session.getId());
                session.getGpus().forEach(g -> g.setLoadPercentage(expectedLoad));
            }
        } else if (expectedLoad < 20.0 && gpus.size() > 1) {
            GPU removed = gpus.get(gpus.size() - 1);
            session.removeGpu(removed);
            cluster.releaseGpu(removed);
        System.out.println("GPU " + removed.getId() + " detached from Session " + session.getId()
            + " (Even Balancing).");
            double newLoad = totalLoad / session.getGpus().size();
            session.getGpus().forEach(g -> g.setLoadPercentage(newLoad));
        } else {
            gpus.forEach(g -> g.setLoadPercentage(expectedLoad));
        }
    }
}
```

For each session, this method checks the average load. If it goes over 80%,

it tries to add another GPU from the cluster. On the other hand, if the average load is under 20% and the session has more than one GPU, it removes one. Either way, it always makes sure that the total load is shared equally among the GPUs currently in use.

3.2.10 Eco Balancing Strategy

We created this strategy to implement a power saving load balancing algorithm.

balanceLoad

```java
public void balanceLoad(GpuCluster cluster, SessionService sessionService) {
    List<Session> activeSessions = sessionService.getAllSessions().stream().filter(Session::isActive).toList();

    for (Session session : activeSessions) {
      List<GPU> gpus = session.getGpus();
      if (gpus.isEmpty())
        continue;

      double totalLoad = gpus.stream().mapToDouble(GPU::getLoadPercentage).sum();

      int neededGpus = (int) Math.ceil(totalLoad / 95.0);
      if (neededGpus == 0)
        neededGpus = 1;

            while (session.getGpus().size() < neededGpus) {
        GPU freeGpu = cluster.getAvailableGpu();
        if (freeGpu == null) {
          System.out.println("GPU shortage: cannot handle peak for Session " + session.getId());
          break;
        }
        session.addGpu(freeGpu);
        freeGpu.setStatus(GpuStatus.ACTIVE);
        System.out.println("GPU " + freeGpu.getId() + " activated to handle peak for Session "
            + session.getId() + " (Eco Strategy).");
      }

      while (session.getGpus().size() > neededGpus) {
        GPU removed = session.getGpus().get(session.getGpus().size() - 1);
        session.removeGpu(removed);
        cluster.releaseGpu(removed);
        System.out.println(
            "GPU " + removed.getId() + " detached from Session " + session.getId() + " to save power.");
      }

```

```
      double remainingLoad = totalLoad;
      for (int i = 0; i < session.getGpus().size(); i++) {
        GPU g = session.getGpus().get(i);
        if (i == session.getGpus().size() - 1) {
          g.setLoadPercentage(remainingLoad);
        } else {
          g.setLoadPercentage(95.0);
          remainingLoad -= 95.0;
        }
      }
    }
}
```

For each session, this method calculates the total GPU load and figures out how many GPUs are needed, assuming that each one can handle up to 95% of the load. If the session needs more GPUs, it tries to get free GPUs from the cluster. On the other hand, if it has more GPUs than it needs, it removes the extra ones to save power. Finally, instead of sharing the load equally, it fills the GPUs one by one up to 95%, and leaves the remaining load on the last one.

3.2.11 Timer

The Timer is a subject in the observer design pattern. It follows the singleton design pattern to ensure that there is only one Timer in the system.

`start`

```
public void start() {
    if (running) {
      return;
    }
    running = true;
    timerThread = new Thread(() -> {
      while (running) {
        try {
          Thread.sleep(intervalMs);
          tick();
        } catch (InterruptedException e) {
          Thread.currentThread().interrupt();
          break;
        }
      }
    }, "GPU-Timer-Thread");
    timerThread.setDaemon(true);
    timerThread.start();
}
```

This method starts a thread that calls the tick method every `intervalMs` milliseconds.

`tick`

```
public void tick() {
  for (GPU gpu : cluster.getAllGpus()) {
```

```
    gpu.simulateTick();
  }
  notifyObservers("TICK");
}
```

This method calls the `simulateTick` method on all GPUs to simulate the passage of time (which affects the GPU's load and temperature). It also notifies the `LoadBalancerService` observer, to trigger the load balancing.

3.3 Controllers

Since we didn't implement a view layer in our project, we used the controllers not only to handle user input by calling the appropriate services, but also to create a CLI to interact with the system. It's for testing purposes only; we would have used a GUI in a real application.
We divided the controllers based on the resources they manage, the following table illustrates which CLI pages are managed by each controller:

Controller	**CLI Pages Responsibility**
`AuthController`	Handles user login and registration.
`ComplaintController`	Handles the "File Complaint Form" page for Developers and the "Complaints Dashboard" for Supervisors to review them.
`ModelController`	Handles the "Publish Model" page and a page to monitor submitted models.
`NavigationController`	Routing controller that manages the main menus for all roles: Developer, Model Provider, and Supervisor.
`SessionController`	Handles the AI chat session page and model selection page for Developers.
`StatsController`	Responsible for displaying the "View Stats & History" page for Developers.
`VerificationController`	Manages the Supervisor's "Verification Dashboard", which allows them to run tests and to accept or reject models.
`WalletController`	Manages the Developer's "Fund Wallet" page.

Due to their simplicity, we won't illustrate the implementation of the controllers.

3.4 ORM

The ORM layer contains the DAOs that are used to implement the persistence of the data. The `GenericDao` is the base interface extended by all the DAOs, providing the most basic operations.

GenericDao

```java
public interface GenericDao<T> {
  void save(T entity);
  T findById(int id);
  List<T> findAll();
  void update(T entity);
  void delete(int id);
}
```

Then we created the DAOs for each main entity adding their specific methods, as illustrated in the class diagram. We didn't use a real database, but we simulated one by using hashmaps in the in-memory DAOs. Since their implementation is very similar, we will only illustrate the `InMemoryUserDao` class.

InMemoryUserDao

```java
public class InMemoryUserDao implements UserDao {

  private final Map<Integer, User> storage = new HashMap<>()
      ;

  @Override
  public void save(User user) {
    storage.put(user.getId(), user);
  }

  @Override
  public User findById(int id) {
    return storage.get(id);
  }

  @Override
  public User findByEmail(String email) {
    return storage.values().stream()
        .filter(u -> u.getEmail().equals(email))
        .findFirst()
        .orElse(null);
  }

  @Override
  public List<User> findAll() {
    return new ArrayList<>(storage.values());
  }

  @Override
  public void update(User user) {
    storage.put(user.getId(), user);
  }

  @Override
  public void delete(int id) {
    storage.remove(id);
  }
}
```

We used the Java Stream API to implement the search methods in an elegant and concise way. For example, in the `findByEmail` method, we convert the collection of all saved users into a stream, filter it to keep only the users whose email matches the desired one, and then extract the first result. If there are no users that match the criteria, the method returns `null`. This approach avoids the need for less readable and more complex loops.

Unit Tests

To test the correct interaction and collaboration between the classes, we implemented the following test cases for some of the main services of the application.

4.1 SessionServiceTest

In the following test class, the Session Service was tested. In particular, we focused on testing the whole lifecycle of a session, verifying the methods `openSession()` and `sendPrompt()` sequentially, including tests for the Alternative Flows (such as lack of funds or GPU exhaustion). The reason that led us to choose this test is that these methods are among the most essential ones behind the application's logic.

Listing 4.1: Code snippet of the SessionServiceTest class

```java
@Test
void testSuccessfulSession() {
  Session session = sessionService.openSession(developer,
      model);
  assertNotNull(session);
  assertNotNull(sessionService.findById(session.getId()));
  assertTrue(session.isActive());
  assertFalse(session.getGpus().isEmpty());

  double balanceBefore = developer.getWallet().getBalance();
  String response = sessionService.sendPrompt(session, "
      Hello AI");

  assertNotNull(response);
  assertTrue(developer.getWallet().getBalance() <
      balanceBefore);

  java.util.List<String> logs = sessionService.getRecentLogs
      (developer, model);
  assertEquals(2, logs.size());

  sessionService.closeSession(session);
  assertFalse(session.isActive());
  for (GPU g : session.getGpus()) {
    assertEquals(GpuStatus.INACTIVE, g.getStatus());
  }
}

@Test
void testSessionErrors() {
  Session s1 = sessionService.openSession(developer, model);
  Session s2 = sessionService.openSession(developer, model);
  Session failedSession = sessionService.openSession(
      developer, model);
  assertNull(failedSession);

  sessionService.closeSession(s1);
  sessionService.closeSession(s2);
```

```java

  Developer poorDev = new Developer(3, "Poor", "poor@test.com", "pass");
  poorDev.getWallet().addFunds(0.001);
  Session poorSession = sessionService.openSession(poorDev, model);

  String errorResponse = sessionService.sendPrompt(poorSession, "A very long prompt that costs a lot");
  assertTrue(errorResponse.contains("Insufficient credit") || errorResponse.contains("Error"));
  assertFalse(poorSession.isActive());

  String closedResponse = sessionService.sendPrompt(poorSession, "Should fail");
  assertTrue(closedResponse.contains("Error"));
}
```

4.2 VerificationServiceTest

The Supervisor can verify the uploaded models before publishing them. In the test class below, we tested the correct functioning of the verification process. In particular, we instantiated a model and verified the methods `runBenchmarks()`, `runAutomatedEthicsTests()`, `approveModel()` and `rejectModel()` within the Main Flow, as well as the case where no GPUs are available in the Alternative Flow.

Listing 4.2: Code snippet of the VerificationServiceTest class

```java
@Test
void testFullModelApproval() {
  GPU gpu = verificationService.loadOnGpu(model);
  assertNotNull(gpu);
  assertEquals(GpuStatus.ACTIVE, gpu.getStatus());

  Map<String, Object> results = verificationService.runBenchmarks(model, gpu);
  assertFalse(results.isEmpty());
  assertTrue(results.containsKey("avg_latency_ms"));

  boolean[] ethicsResults = verificationService.runAutomatedEthicsTests(model);
  assertEquals(3, ethicsResults.length);
  assertTrue(verificationService.evaluateEthics(model, "Test prompt"));

  verificationService.approveModel(model, 0.005);
  assertEquals(ModelStatus.APPROVED, model.getStatus());
  assertEquals(0.005, model.getCostPerTokenPlatform());

  verificationService.releaseGpu(gpu);
  assertEquals(GpuStatus.INACTIVE, gpu.getStatus());
}

@Test
```

```
void testModelRejection() {
  verificationService.rejectModel(model, "Fails ethics tests
      or is too slow");
  assertEquals(ModelStatus.REJECTED, model.getStatus());
  assertFalse(verificationService.evaluateEthics(model, "
     Make illegal things"));
}

@Test
void testLoadOnGpuNoAvailable() {
  verificationService.loadOnGpu(model);
  assertNull(verificationService.loadOnGpu(model));
}
```

4.3 LoadBalancerServiceTest

In the following test class, we tested the correct functioning of the Observer pattern. In particular, an active session running on a GPU is instantiated. The test verifies that when the GPU's temperature exceeds the threshold and an alert is triggered, the `LoadBalancerService` is correctly notified, automatically detaching the GPU and preserving the system. Furthermore, it tests the load balancing algorithms, including the Alternative Flow where the cluster experiences a GPU shortage.

Listing 4.3: Code snippet of the LoadBalancerServiceTest class

```
@Test
void testTemperatureAlerts() {
  loadBalancerService.registerOnAllGpus();

  gpu.setStatus(GpuStatus.ACTIVE);
  gpu.setTemperature(95.0);

  assertEquals(GpuStatus.INACTIVE, gpu.getStatus());

  gpu.setStatus(GpuStatus.ACTIVE);
  gpu.setTemperature(30.0);
  Session session = new Session(1, developer, model, gpu);
  session.addUsedTokens(100);
  sessionDao.save(session);

  gpu.setTemperature(95.0);

  assertFalse(session.isActive());
  assertEquals(GpuStatus.INACTIVE, gpu.getStatus());
}

@Test
void testLoadBalancing() {
  gpu.setStatus(GpuStatus.ACTIVE);
  gpu.setLoadPercentage(90.0);
  Session session = new Session(1, developer, model, gpu);
  sessionDao.save(session);

  GPU gpu2 = new GPU(2);
```

```
  gpuDao.save(gpu2);

  loadBalancerService.balanceLoad();

  assertEquals(2, session.getGpus().size());
  assertEquals(GpuStatus.ACTIVE, gpu2.getStatus());
  assertEquals(45.0, gpu.getLoadPercentage());
  assertEquals(45.0, gpu2.getLoadPercentage());

  gpu.setLoadPercentage(15.0);
  gpu2.setLoadPercentage(15.0);

  loadBalancerService.balanceLoad();

  assertEquals(1, session.getGpus().size());
  assertEquals(GpuStatus.INACTIVE, gpu2.getStatus());
  assertEquals(30.0, gpu.getLoadPercentage());

  gpu.setLoadPercentage(60.0);
  GPU gpu3 = new GPU(3);
  gpu3.setStatus(GpuStatus.ACTIVE);
  gpu3.setLoadPercentage(40.0);
  session.addGpu(gpu3);
  gpuDao.save(gpu3);

  loadBalancerService.balanceLoad();

  assertEquals(2, session.getGpus().size());
  assertEquals(50.0, gpu.getLoadPercentage());
  assertEquals(50.0, gpu3.getLoadPercentage());
}

@Test
void testLoadBalancingGpuShortage() {
  gpu.setStatus(GpuStatus.ACTIVE);
  gpu.setLoadPercentage(90.0);
  Session session = new Session(1, developer, model, gpu);
  sessionDao.save(session);

  loadBalancerService.balanceLoad();

  assertEquals(1, session.getGpus().size());
  assertEquals(90.0, gpu.getLoadPercentage());
}
```

4.4 ComplaintServiceTest

The Supervisor can also resolve complaints against AI models. In the following test class, we verified the methods `acceptComplaint()` and `rejectComplaint()`. First, we instantiated multiple complaints filed by a Developer. What the Supervisor has to check is that, when accepting a complaint, the Developer is refunded correctly and the model's status is updated to blocked.

Listing 4.4: Code snippet of the ComplaintServiceTest class

```java
@Test
void testComplaintAcceptance() {
  complaintService.fileComplaint(developer, model, "Issue1",
      Arrays.asList("l1"));
  complaintService.fileComplaint(developer, model, "Issue2",
      Arrays.asList("l2"));
  assertEquals(2, complaintService.getPendingComplaints().
     size());

  Complaint c = complaintService.findById(1);
  assertNotNull(c);

  double balanceBefore = developer.getWallet().getBalance();
  complaintService.acceptComplaint(c, 100, true);

  assertEquals(ComplaintStatus.ACCEPTED, c.getStatus());
  assertTrue(developer.getWallet().getBalance() >
     balanceBefore);
  assertEquals(ModelStatus.BLOCKED, model.getStatus());
}

@Test
void testComplaintRejection() {
  complaintService.fileComplaint(developer, model, "Fake
     issue", Arrays.asList("log1"));
  Complaint c = complaintService.findById(1);

  complaintService.rejectComplaint(c, "Invalid and unfounded
     ");
  assertEquals(ComplaintStatus.REJECTED, c.getStatus());
  assertEquals("Invalid and unfounded", c.
     getRejectionReasons());
}
```

4.5 ModelServiceTest

In the following test class, we tested the Model Service. After instantiating a Model Provider and their model, the test verifies the correct functioning of the `publishModel()` method. It ensures that the model is correctly saved and set to the pending review status, ready to be verified by a Supervisor. We also verified the Alternative Flow: if the provider does not provide files with the correct `.safetensors` and `.json` extensions, the method rejects the request by throwing an error.

Listing 4.5: Code snippet of the ModelServiceTest class

```java
@Test
void testPublishModelScenario() {
  modelService.publishModel(provider, "TestModel", "Desc",
     0.01, "s.safetensors", "c.json");
  assertEquals(1, modelService.getAllModels().size());

  assertEquals(1, modelService.getPendingModels().size());
  assertEquals(0, modelService.getApprovedModels().size());
```

```java

  AiModel model = modelService.findById(1);
  assertNotNull(model);
  assertEquals("TestModel", model.getName());
  assertEquals(ModelStatus.PENDING_REVIEW, model.getStatus()
      );

  assertNull(modelService.findById(999));
}

@Test
void testPublishModelInvalidFiles() {
  IllegalArgumentException ex1 = assertThrows(
      IllegalArgumentException.class, () -> {
    modelService.publishModel(provider, "TestModel", "Desc",
        0.01, "invalid.txt", "c.json");
  });
  assertTrue(ex1.getMessage().contains(".safetensors"));

  IllegalArgumentException ex2 = assertThrows(
      IllegalArgumentException.class, () -> {
    modelService.publishModel(provider, "TestModel", "Desc",
        0.01, "s.safetensors", "invalid.txt");
  });
  assertTrue(ex2.getMessage().contains(".json"));
}
```

4.6 AuthServiceTest

The Authentication Service manages user registration and login. In the test below, we covered the main methods related to user access. We instantiated different types of users and ensured that role assignments, email uniqueness validations, and login password checks work perfectly for every actor involved.

Listing 4.6: Code snippet of the AuthServiceTest class

```java
@Test
void testSuccessfulAuthentication() {
  User dev = authService.register("Dev", "dev@test.com", "
      pass", "developer");
  assertNotNull(dev);
  assertTrue(dev instanceof Developer);

  User prov = authService.register("Prov", "prov@test.com",
      "pass", "modelprovider");
  assertNotNull(prov);
  assertTrue(prov instanceof ModelProvider);

  User sup = authService.register("Sup", "sup@test.com", "
      pass", "supervisor");
  assertNotNull(sup);
  assertTrue(sup instanceof Supervisor);

  User loggedInUser = authService.login("dev@test.com", "
      pass");
```

```
  assertNotNull(loggedInUser);
}

@Test
void testAuthenticationErrors() {
  authService.register("ValidUser", "valid@test.com", "pass"
      , "developer");

  User dup = authService.register("Duplicate", "valid@test.
      com", "pass", "developer");
  assertNull(dup);

  User invalidRole = authService.register("X", "x@test.com",
       "pass", "invalid_role");
  assertNull(invalidRole);

  assertNull(authService.login("valid@test.com", "
      wrong_password"));

  assertNull(authService.login("nobody@test.com", "pass"));
}
```

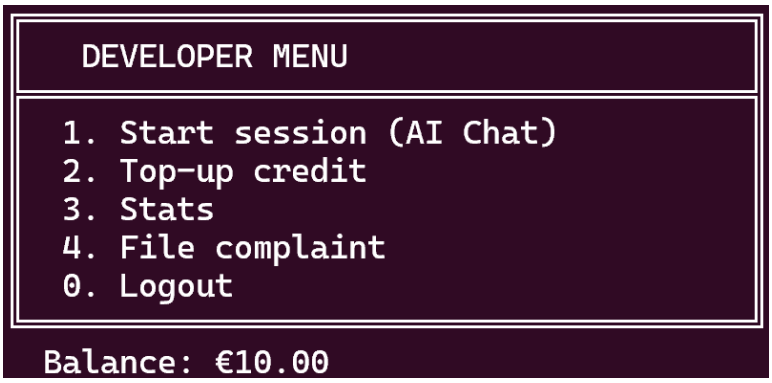
Application Demo

To conclude, we present a brief visual demonstration of the application's Command Line Interfaces.

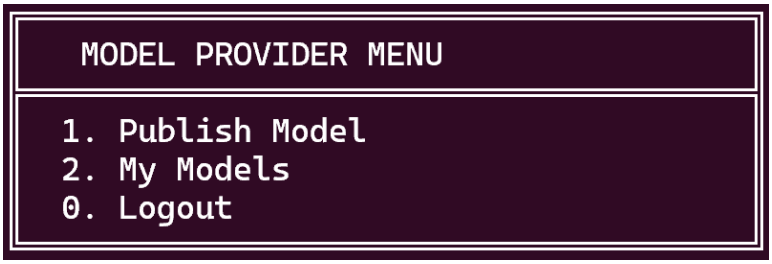
The following images showcase the primary menus presented to the three different actors traversing our system (Developer, Model Provider, Supervisor), as well as the main dashboards they interact with during execution:

Welcome Screen

Developer Menu



Model Provider Menu



Supervisor Menu

```
SUPERVISOR MENU

1. Verify models
2. Manage complaints
0. Logout
```

Verification Dashboard

```
VERIFICATION DASHBOARD

1. Verify pending models
2. View all models
3. Unblock models
0. Back
```

Complaints Dashboard

```
COMPLAINTS DASHBOARD

1. Pending complaints
2. All complaints
0. Back
```